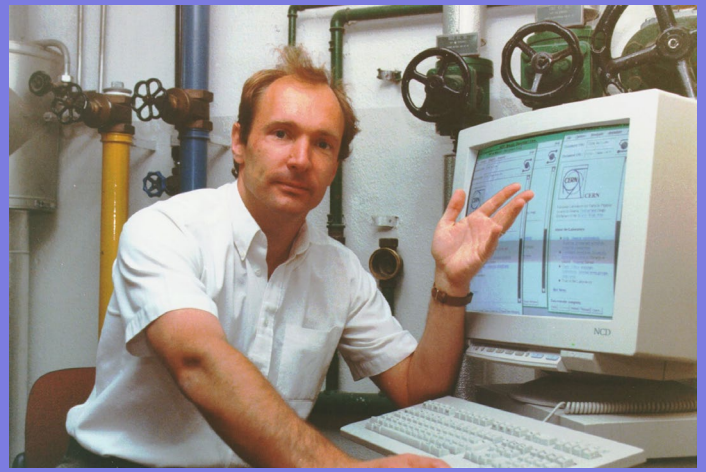
Web Basics &HTML

Web-Design und Programmierung

“The World Wide Web (W3) is a wide-area hypermedia information retrieval initiative aiming to give universal access to a large universe of documents.”

Tim Berners Lee



“Hypertext is a corpus of written or visual material that is linked in such a complex way that it would be very difficult to present or display on paper.”



Ted Nelson

What's the Difference?



HTML
Hypertext Markup Language

Create the structure
- Controls the layout of the content
- Provides structure for the web page design
- The fundamental building block of any web page



CSS
Cascading Style Sheet

Stylize the website
- Applies style to the web page elements
- Targets various screen sizes to make web pages responsive
- Primarily handles the "look and feel" of a web page



Javascript

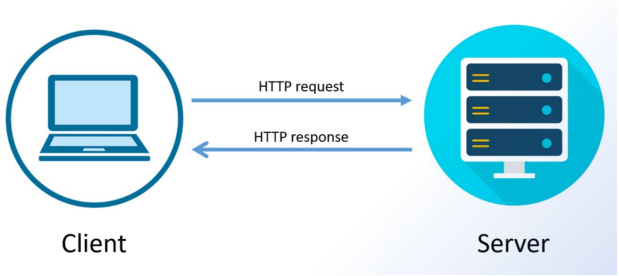
Increase interactivity
- Adds interactivity to a web page
- Handles complex functions and features
- Programmatic code which enhances functionality

How does the Web work?

- World Wide Web
- Request Response Model
- HTTP

World Wide Web

- Web is a subset of the Internet
- Web consists of **pages** which can be **requested** from **servers** and viewed with a browser
- Web contains of multiple protocols/technologies like HTML, CSS, JS, HTTP(S), …
- Request/Response communication model
  - Client sends request
  - Server processes request and returns response
  - No “push messages” from server



Web server

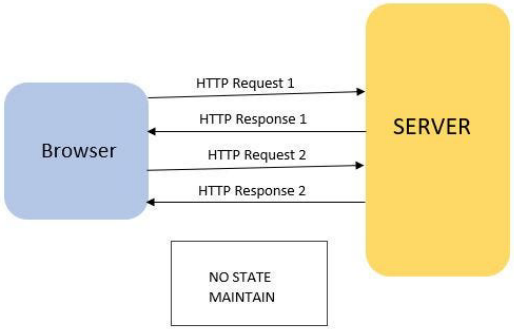
- Web server: Software that provides information via HTTP/HTTPS (port 80/443)
- Web server directory
  - Default directory of the domain (htdocs in case of Apache)



HYPERTEXT TRANSFER PROTOCOL



- De facto standard protocol for communication on the web
- **Text-based** (= non-binary)
- Request/response concept
  - Client sends a request to the server
  - Server sends response back to client
- **Stateless**
  - Server handles all client requests completely independently of each other
- **Connectionless**
  - No permanent connection between client and server required



HyperText Transfer Protocol - Request

- **Request**

  ```
  GET /campus-hagenberg HTTP/1.1
  Host: www.fh-ooe.at
  […]
  ```

- Request:
  - Method („Verb“)
  - Request-URI
  - Protocol version
- Header
- Blank line (obligatory!)
- Body (optional)



Hypertext Transfer Protocol - Request

- **Verbs**
  - Describes action to be taken for resource identified via URL
- HTTP consists of eight verbs

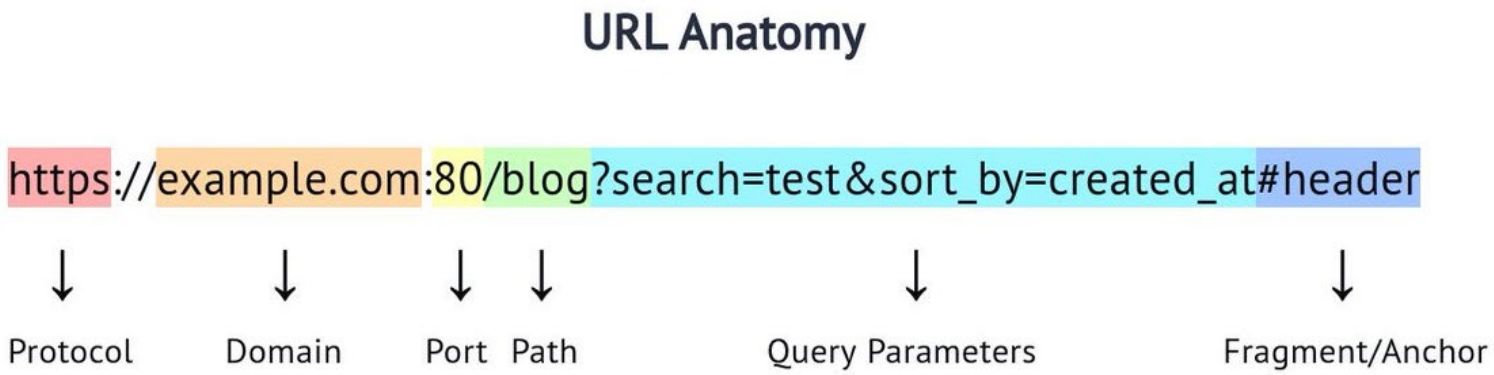
- **GET**: request a resource
- **POST**: create a new resource
- **PUT**: edit a resource
- **DELETE**: delete a resource from server

- **OPTIONS**: request information for communication path to a resource
- **HEAD**: request a resource, only Header
- **TRACE**: For test purposes, request message is returned in the body of the response
- **CONNECT**: Tunneling with HTTP-Proxies

Uniform Resource Identifier

- Identifies affected resource
- Absolute path, starting with "/"
- Interpreted by the server and reallocated to the corresponding resource

- Examples:
  - “/”
  - “/index.html”
  - “/images/logo.png”
  - “/research/staff/12”



HyperText Transfer Protocol - Response

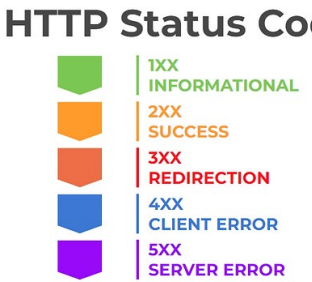
- **Status line**, consisting of:
  - Protocol version
  - Status code
  - Status message
- **Header**
- **Blank line**(obligatory!)
- **Body**(optional)

```
HTTP/1.0 200 OK
Header-4: Value of header 4
Header-5: Value of header 5
```

This data here is called the body of the response and is transferred back to the client. It is again optional.

HyperText Transfer Protocol – Status Codes

- Three-digit number (standardized)
- For machine processing of the response
- Indicates whether the server understood the request message and was able to process it.



- **1xx: Information**
  - received Request, Processing (still) in progress
- **2xx: Success**
  - Request received, understood and processed
- **3xx: Forwarding**
  - Request received, additional action required from client
- **4xx: Error on client**
  - Request syntactically incorrect or not valid in terms of content
- **5xx: Error on server**
  - Request valid, error during processing by the server

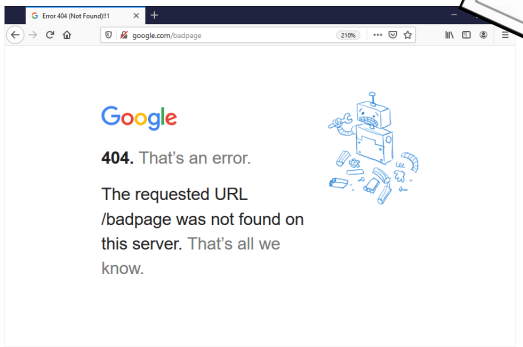
Hypertext Transfer Protocol – Status Codes

- **HTTP/1.1 400 Bad Request**
  - Syntactically incorrect request message
- **HTTP/1.1 401 Unauthorized**
  - Request requires authentication
- **HTTP/1.1 403 Forbidden**
  - Server has understood request, but refuses execution
- **HTTP/1.1 404 Not Found**
  - Resource is not available
- **HTTP/1.1 405 Method Not Allowed**
  - Used method ("verb") is not supported for the requested resource
- **HTTP/1.1 500 Internal Server Error**
  - Unexpected server error occurred during processing







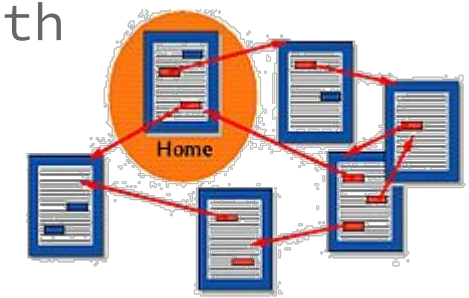


Hypertext Markup Language

- Hypertext
- HTML Elements and attributes
- Basic elements

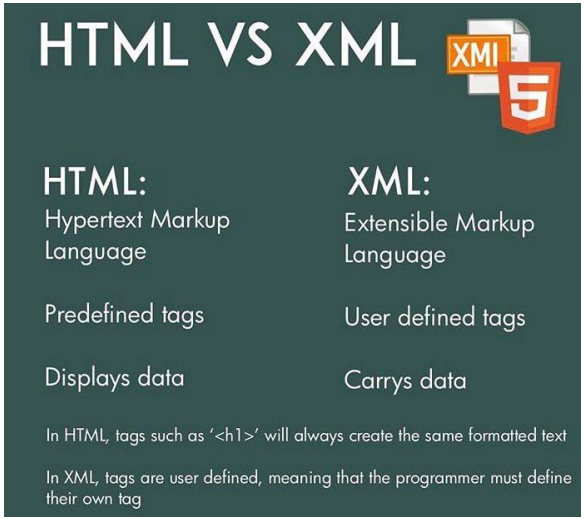
Hypertext

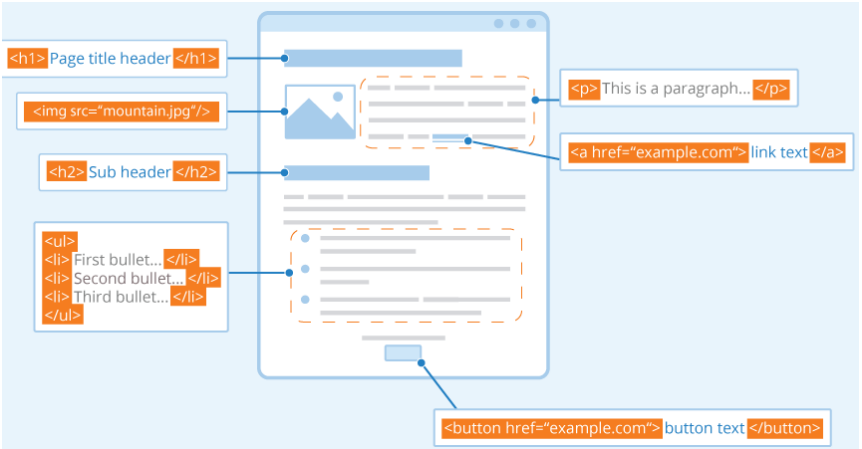
- Non-linear organization of objects whose net-like structure is created by **logical links between knowledge units** (nodes, e.g., texts or text parts) (reference node concept)
- Hypertext and hypermedia are often used synonymously
- **Hypermedia** often refers to hypertext with multimedia aspects
  - this is the norm today



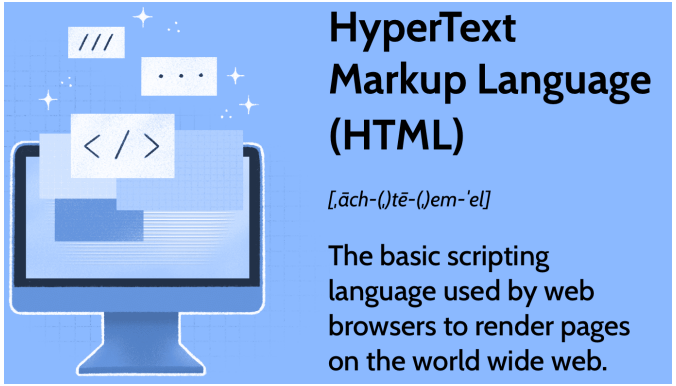
Markup Language

- Describes data or procedures for representation
  - Descriptive markup languages
    - Describes **syntax** of data
      - e.g., HMTL, XML
  - Procedural markup languages
    - Describes the procedure for generating a representation
      - e.g., TeX, PDF
- Description with “tags”
  - 145 predefined ones in HMTL5

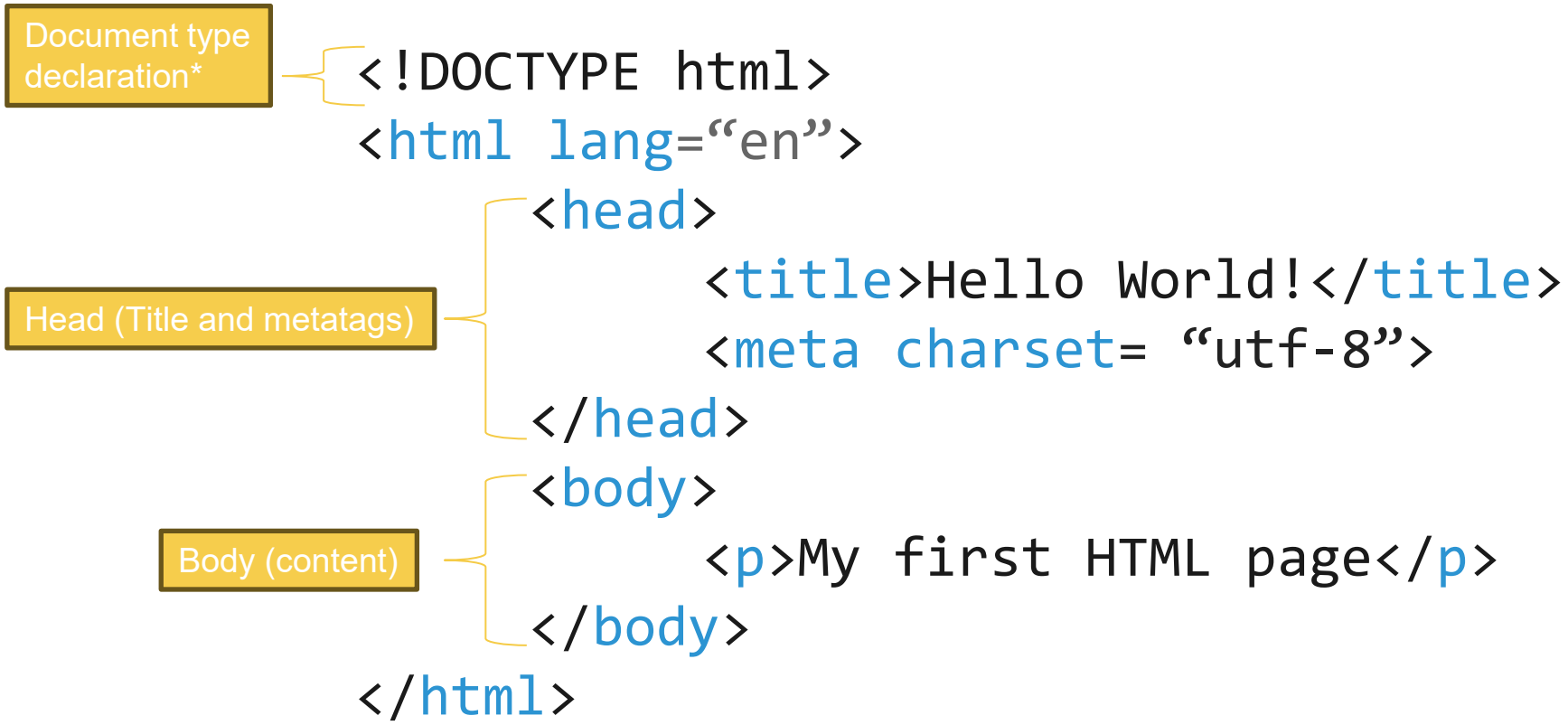




HyperText Markup Language



- used to **structure the content** of a web page
- **media-independent** description of interactive content
- HTML holds everything together
  - content, CSS files, JS files, images, …
- HTML user agents (e.g., browsers) **parse** the markup line by line, turning it into a **DOM** (Document Object Model) tree, which is an in-memory representation of a document.

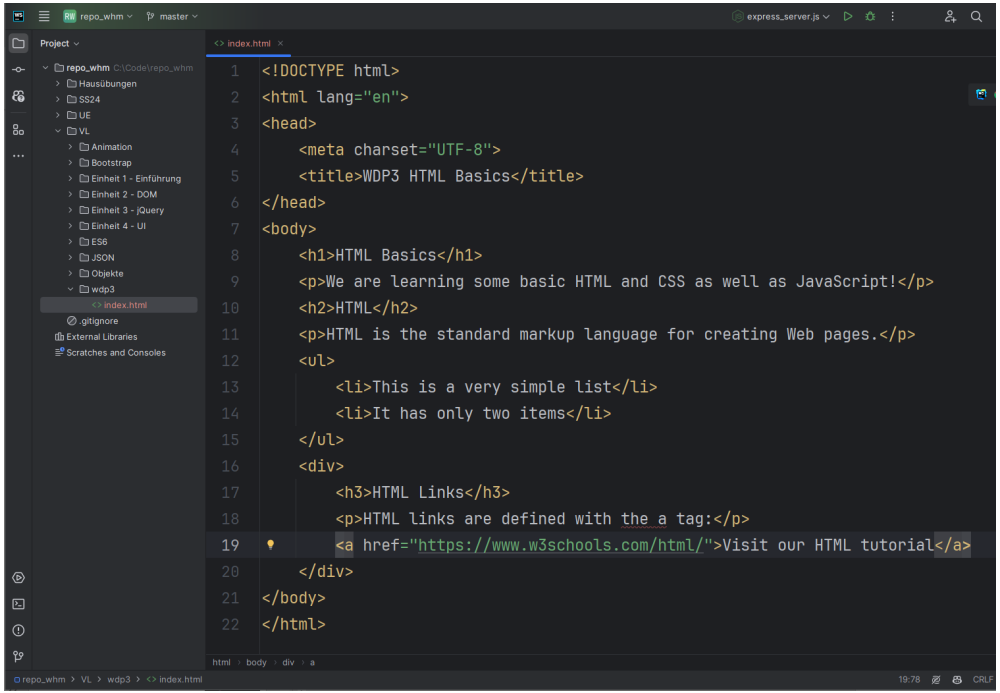


*List of valid doctype declarations: https://www.w3.org/QA/2002/04/valid-dtd-list.html

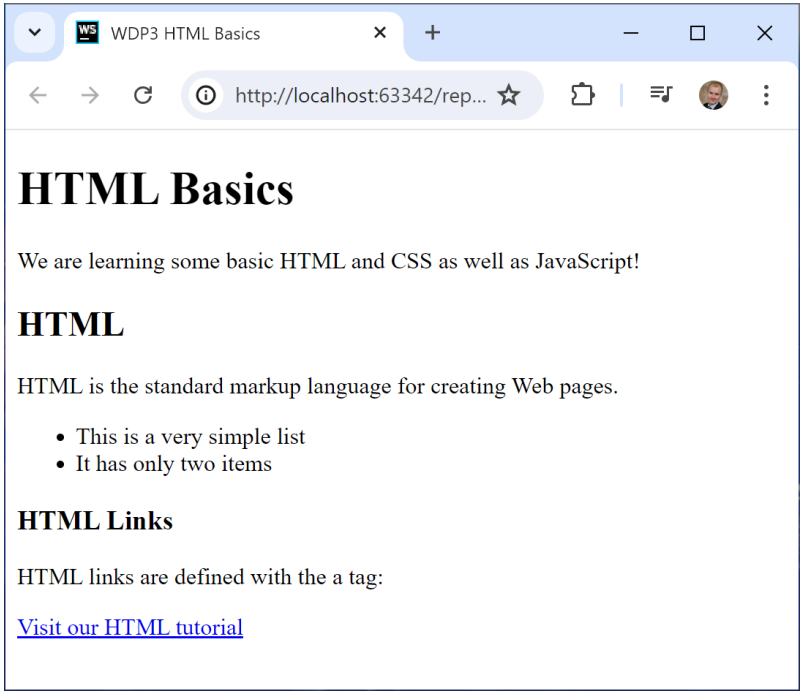
HyperText Markup Language

- HTML documents consist of a tree of elements and text.
- Elements have a start tag, such as , and an end tag, such as .
- Tags have to be nested, such that elements are completely within each other.
- Everything between a start tag and an end tag is called: content of an element.

```
<p>This is some <strong>highlighted</strong> text.</p> 🙂
<p>Please never do <strong>this!</p></strong> 😟
```

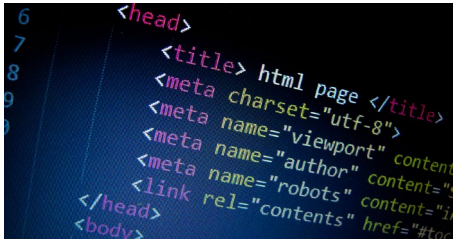


Text Editor
(Webstorms)



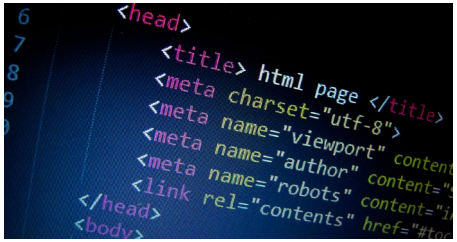
Browser
(Google Chrome)

HyperText Markup Language - Header



- General data on document
  - Description of content and additional information
  - <meta name=“property" name=“value" />
  - E.g., <meta name=“author" name=“maxmustermann" />
    - **description**
    - **keywords** (google and web ranking)
    - **date**
    - Instructions for crawlers: **robots** (follow, index, nofollow, noindex, …)
    - Inclusion of Stylesheets (good) or JavaScript (bad)
    - https://developer.mozilla.org/en-US/docs/Web/HTML/Element/meta

HyperText Markup Language - Header

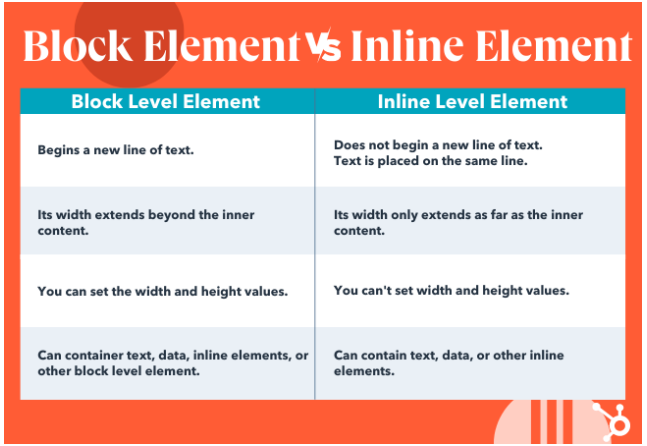


- Web page should contain the following meta information
  - language of the page
  - character encoding of the document.

- The language is defined as lang-attribute of the html-tag:
  - `<html lang="en">` or `<html lang="de">`
- The character set is defined using a meta tag in the page's head section:
  - `<meta charset="UTF-8" />`
  - Inclusion of special characters (e.g., Emojis) or HTML Entities
    - https://www.w3schools.com/charsets/ref_emoji_smileys.asp

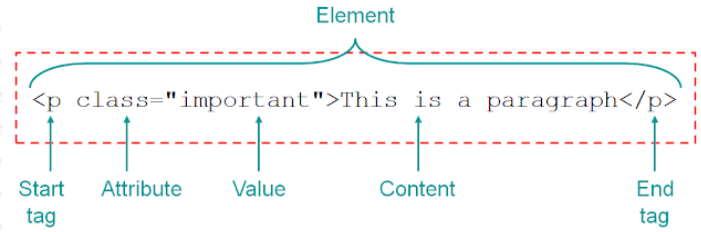
HyperText Markup Language – HTML Elements

- HTML Body contains the content Elements (**HTML Tags**)
- Types of HTML Elements
  - **Block-Elements** (e.g.,)
    - Elements start in a new line
    - Structure the content; position can be defined
    - Can contain inline and block elements
    - 100 % width, height according to content
  - **Inline-Elements**
    - Wrap small parts of content
    - Width according to content, height according to line height
  - **Void-Elements or self-closing elements**
    - Elements without “content”
    - E.g.,
, <hr>
    - Trailing slash is optional
  - **Text**
- **Hierarchically structured**
  - https://developer.mozilla.org/en-US/docs/Web/HTML/Element
  - https://html.spec.whatwg.org/#kinds-of-content



HyperText Markup Language – Attributes

- Marking objects with properties
  - `<h1 align="center">centered headline</h1>`
- All elements can have attributes
  - Some require specific attributes to actually work (e.g., src-Attribute for `img`)
- Always specified in the **start tag**
- **Types of attributes**
  - Value assignment with predefined values
    - `<input disabled="disabled"/>`
  - Free value assignment, but with data type
    - `<input size="12"/>`
  - Completely free value assignment
    - `<p title="foo">`
- **Important attributes**
  - id-attribute -> Unique identifier of an Element
  - name-attribute -> Needs not to be unique (used for grouping)
  - Class -> assignment of CSS classes



HyperText Markup Language – Important General Tags

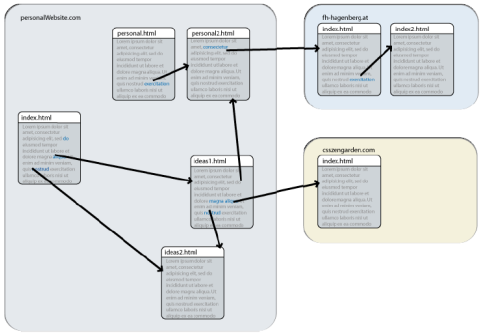
- Headlines
  - <h1>…</h1> - <h6>…</h6>
- Paragraph
  - …
  - May not contain other block elements
- Lists
  - … (unordered list), … (ordered list)
  - … (list item)
  - Change enumeration item
    - For ol -> type=“val”, val can be either **I,i,A,a,** or **1** (default 1)
    - For ul type is deprecated (use CSS list-style-type)
  - Can be hierarchically structured

Hypertext Markup Language – Important General Tags



- **Line break </br>**
  - Explicit line breaks need to be specified
  - Do not “cascade” line breaks -> use CSS
- **Semantic text**
  - …, …,<code>…</code>, <samp>…</samp>, <kbd>…</kbd>, <var>…</var>, <cite>…</cite>, <dfn>…</dfn>, <abbr>…</abbr>, …, …, <u>…</u>,<small>…</small>, <big>…</big>
  - … generic inline container; no inherent semantic meaning; typically used together with CSS/classes
  - see https://wiki.selfhtml.org/wiki/HTML/Tutorials/Textauszeichnung for details
- **Principle**
  - **HTML → structure & meaning**
  - **CSS → presentation**
  - **Example:** Warning! better than Warning!

HyperText Markup Language – Important General Tags



- **Links** <a>…
  - link from one to another HTML document
  - link to an anchor within the same page
  - link to an external resource
  - Requires href Attribute with the links target
    - `<a href="index.html">This is the anchor text</a>`
- **Reference**
  - **Absolute** (remote or local)
    - Protocol://server.domainname/filename.extension
      - `<a href="http://www.duckduckgo.com">Link to DuckDuckGo</a>`
  - **Relative**
    - `<a href="intro.html">Introduction</a>`

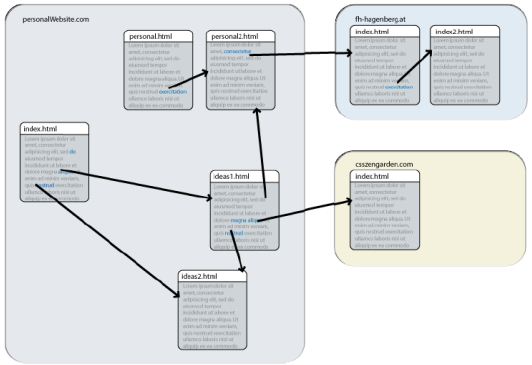
HyperText Markup Language – Important General Tags

- **Links** (cont.)
  - Target-Attribute defines how new page should be presented
  - _self, _parent, _blank, _top
  - Mostly relevant for (old concept of) frames
    - _blank to open new browser tab

  - Reference in same document (Anchor)
    - A-tag points to id of an element
    - `<a href="file.html#anchor">Link to anchor</a>`

      …

      `<h2 id="anchor">Introduction</a>`
  - E-Mail
    - `<a href="mailto:test@gmail.com">Mail</a>`



Hypertext Markup Language – Important General Tags

- **Horizontal rule** <hr/>
- **Images** <img/>
  -
  - Src -> path to image
  - Alt -> Alternative text (accessibility)
  - Check for filetypes
    - https://developer.mozilla.org/en-US/docs/Web/Media/Formats/Image_types
- **Images** can be wrapped with <figure>…</figure> tag
  - Allows for according caption

```
<figure>
  <img src="favicon-192x192.png" alt="The beautiful MDN logo." />
  <figcaption>MDN Logo</figcaption>
</figure>
```

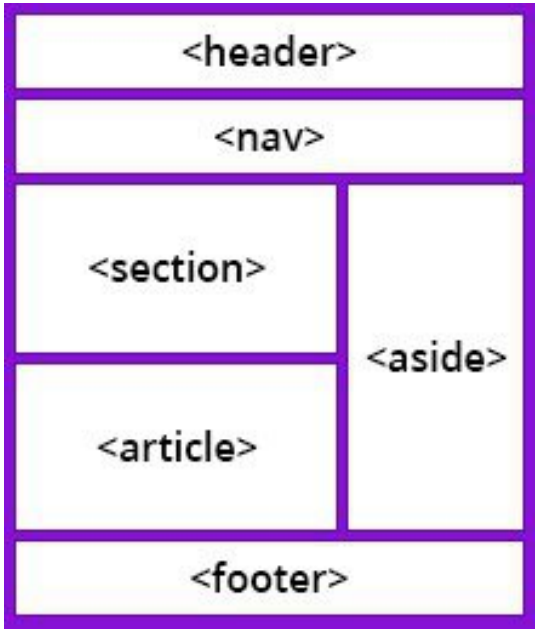
HyperText Markup Language – Important General Tags

- <picture>…</picture> - Responsive images
  - Contains one or more <source> elements and **one <img> fallback**
  - Browser checks <source> elements in order
  - media and type determine whether a <source> is applicable
  - srcset provides one or more image candidates
  - If no <source> matches, the <img src="…"> is used
  - The <img> element **displays the selected image**
  - **Be aware: img** also provides srsct -> used for the same image, but differents sizes (cf. Responsive design)

```
<picture>
  <source srcset="/media/cc0-images/surfer-240-200.jpg"
          media="(orientation: portrait)" />
  <img src="/media/cc0-images/painted-hand-298-332.jpg" alt="" />
</picture>
```

HyperText Markup Language – Important General Tags

- Elements to (logically) structure page content
  - Document Division Element
  - Sections
    - Header, Footer <header>, <footer>
    - Section <section>, <article>
    - Sidebar <aside>
    - Navigation <nav>
    - Main <main>



HyperText Markup Language – Comment

```
<!-- There is nothing more to say here. -->
```

HTML Tables

- Table
- Table header/body
- Table row/data

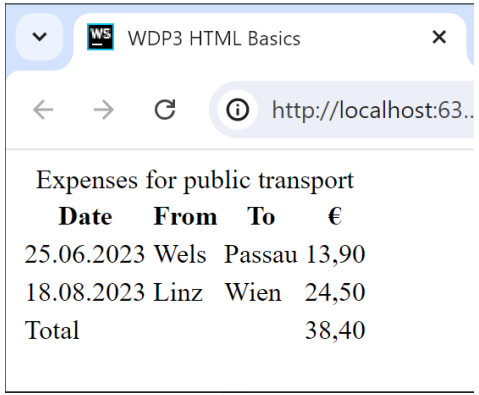
HTML - Tables

- Tables <table> are for structuring data
  - Do not use them for layout

- Optional table header <thead>,body <tbody> and footer <tfoot>
- Table rows <tr> and columns <td>
  - For header use <th> (instead of <td>)
- Aggregation of rows/cols using rowspan/colspan attributes
- The (optional) <caption> element is always the first child of a table

```
<table>
  <caption>Expenses for public transport</caption>
  <thead>
  <tr>
    <th>Date</th>
    <th>From</th>
    <th>To</th>
    <th>€</th>
  </tr>
  </thead>
  <tbody>
```

```
 <tr>
        <td>25.06.2023</td>
        <td>Wels</td>
        <td>Passau</td>
        <td>13,90</td>
    </tr>
    <tr>
        <td>18.08.2023</td>
        <td>Linz</td>
        <td>Wien</td>
        <td>24,50</td>
    </tr>
    </tbody>
    <tfoot>
    <tr>
        <td
colspan="3">Total</td>
        <td>38,40</td>
    </tr>
    </tfoot>
</table>
```



HTML Forms

- Form
- Form elements
- Validation and UX

HTML Forms

- Interaction between the user and an application
- Collect and send data to a web server
- Common use cases: login form, contact form, newsletter subscription, conference registration, ordering process, search form, filter…

- Client side => create forms, how to send data
- Server side => receive and process data

HTML Forms

```
<form action="…" method="post" />
<!-- form elements -->
</form>
```

Form action & method

bei GET: in URL werden Paarameter mitgesendet - d.h. ungeeignet für logins
POST: Files werden dem Body angehängt - für sensible Daten oder auch binärfiles

- Action: URL where form data is sent on submit
- Method: HTTP method to send data with
  - GET: to ask the server for data, to get data
    - Form data is appended to the URL
    - (key-value pairs) => can be bookmarked, will show up in server logs
    - Length of the URL is limited
    - Values are transmitted openly
    - Special characters must be encoded/escaped
    - Use for: search form, filter, pagination,…
    - Example: https://www.smashingmagazine.com/search/?q=html
  - POST: to send data to the server
    - Form data is sent via request body
    - Transmitted “invisibly” for the user
    - Own data block - not limited in size like GET
      - Data (files, images, etc.) can also be sent
    - use for: login, contact form, registration,…

Input & label

- The <input> element is the most common element in a form
- Add the type attribute to specify its behavior
  - text, email, password, number, date, checkbox, radio, …
- Add the <label> element to explain the input field
  - The input field's id and the label's for attribute must match

```
<label for="name-input">Enter your username</label>
<input type="text" id="name-input" name="username" />
```

Input Type

- Inputs with type **text** create basic, single line inputs
- Other types:
  - “date”: Date
  - “datetime”: Date + Time
  - “email”: eMail address
  - “tel”: Phonenumber
  - “number”: Numerical value
  - “color”: Color chooser
  - “range”: Interval
  - “url”: URL

Radio Buttons

- type="radio"
- Radio buttons are generally used in a set of related options where only one of the options can be selected at a time.
- Grouped radio buttons must have the same **name** attribute, the value is used to identify the selected option.

```
<input type="radio" name="campus" id="hagenberg" value="hagenberg" />
<label for="hagenberg">Hagenberg</label>
<input type="radio" name="campus" id="linz" value="linz" checked />
<label for="linz">Linz</label>
<input type="radio" name="campus" id="steyr" value="steyr" />
<label for="steyr">Steyr</label>
<input type="radio" name="campus" id="wels" value="wels" />
<label for="wels">Wels</label>
```

Checkboxes

- type="checkbox"
- Checkboxes are used to turn individual values on or off
- In contrary to radio buttons more than one value can be selected in a checkbox group.

```
<input type="checkbox" name="interests" id="music" value="music" />
<label for="music">Music</label>
<input type="checkbox" name="interests" id="art" value="art" />
<label for="art">Art</label>
<input type="checkbox" name="interests" id="tech" value="tech" />
<label for="tech">Tech</label>
<input type="checkbox" name="interests" id="cooking" value="cooking" />
<label for="cooking">Cooking</label>
```

Input Attributes

- type: the input’s type
- name: value’s key when data is sent to server
- value: the value sent to the server
- id: to identify the input and associate a label with the input
- placeholder: a placeholder value, visible if nothing has been entered
- required: whether the input field is required to send the form data
- disabled: whether the input field should be disabled
- Min/max: Min/max value for numerical values

Select (Dropdown)

- The select element (sometimes called dropdown) <select> is used to select a value from a number of options <option>
- Options can be grouped using the <optgroup> tag
- Attribute multiple allows for multiselect (Strg + Click)

```
<label for="countries">Choose your country</label>
<select id="countries">
    <optgroup label="Europe">
        <option value="austria">Austria</option>
        <option value="germany">Germany</option>
        <option value="italy">Italy</option>
    </optgroup>
    <optgroup label="USA">
        <option value="newyork">New York</option>
        <option value="california">California</option>
        <option value="texas">Texas</option>
    </optgroup>
</select>
```

Datalist



- New form element
  - <datalist> Suggestion list for inputs

```
<input type="text" list="web"/>
<datalist id="web">
    <option value="HTML"/>
    <option value="CSS"/>
    <option value="PHP"/>
    <option value="Javascript"/>
</datalist>
```

Textarea

- A <textarea> allows for multiline input
- A <textarea> is used to let the user enter some free-form text, e.g. a message in a contact form or an additional comment.
- Use the rows and cols attributes to specify a size.

```
<textarea name="test" cols="30" rows="4"></textarea>
```

Fieldset

- A <fieldset> is used to group several form elements and labels within a form.
- A fieldset can have a nested <legend> attribute providing a caption for the fieldset.
- When a fieldset is disabled, all form controls within the fieldset are disabled.

```
<form>
    <fieldset>
        <legend>Anrede:</legend>
        <input type=“radio“ name=“gender“ value=“male“ />männlich
        <input type=“radio“ name=“gender“ value=“female“ />weiblich
    </fieldset>
</form>
```

Buttons

- Click-Button

```
<button type="button" name="Textbutton1" onclick="...">clickme</button>
```

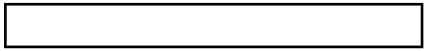
- Submit-Button
  - submits form-data

```
<form action="index.php" method="post" id="form1">
  ...
  <button type="submit" form="form1" value="Submit">Submit</button>
</form>
```

- Reset-Button
  - resets all form values to its initial values

```
<button type="reset" value="Reset">Reset</button>
```

Button vs Input type="Button"



Button

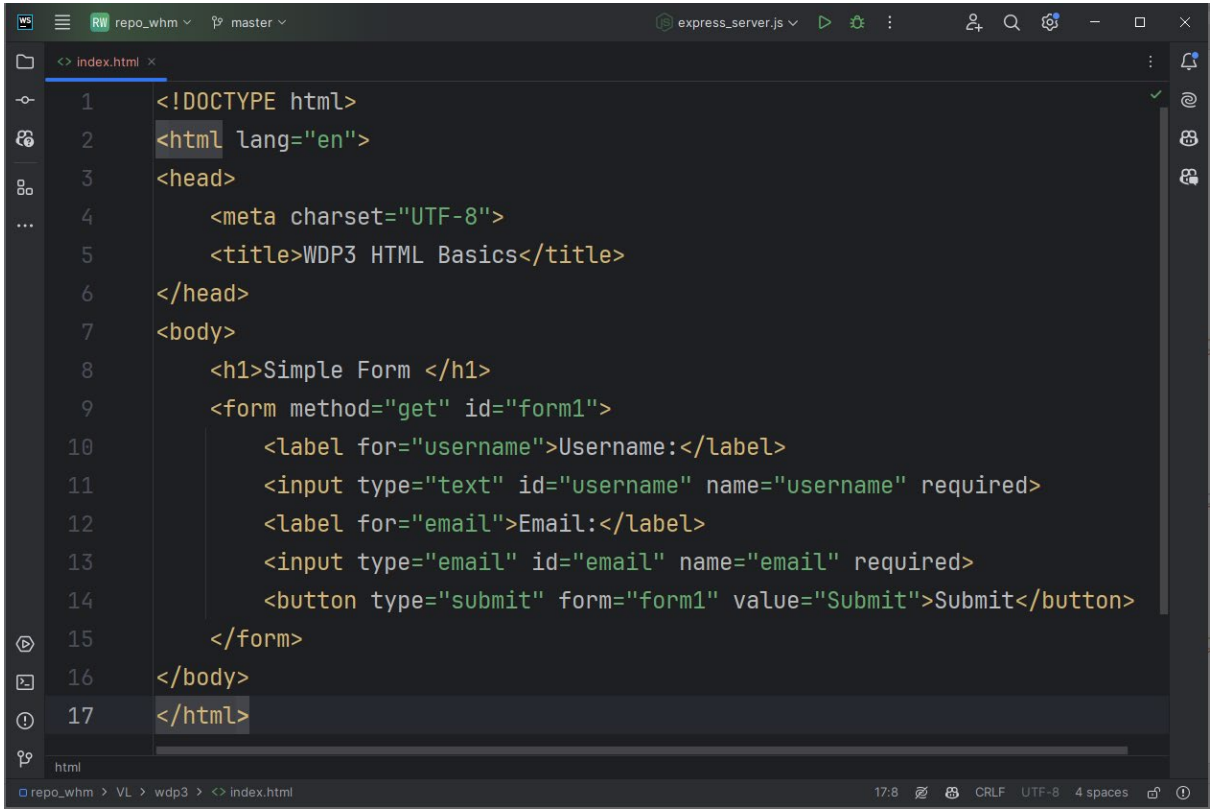
- by default behaves like if it had a "type="submit" attribute
- can be used without a form as well as in forms.
- text or html content allowed
- css pseudo elements allowed (like :before)



Input type

- type should be set to 'submit' to behave as a submitting element
- can only be used in forms
- only text content allowed
- no css pseudo elements

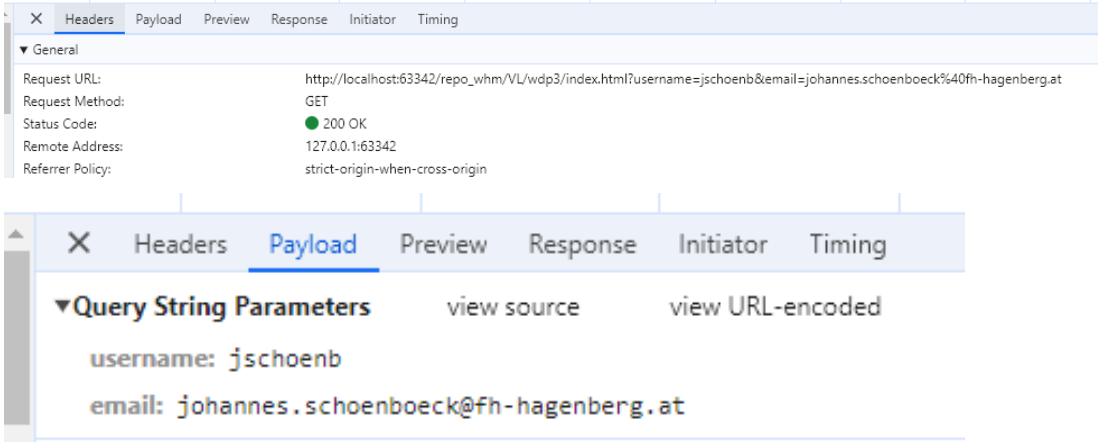
Form Submission



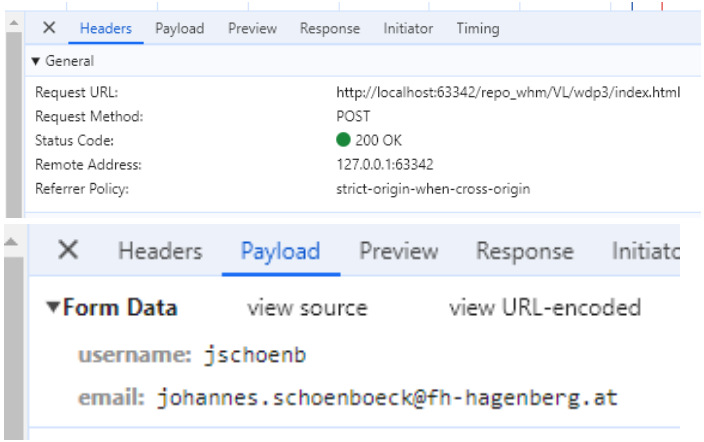
Form Submission

- If no action is specified, data is sent to the current page
- Use Chrome DevTools to find out what data is sent by the form

method="GET"



method="POST"



Form Validation

- **Never trust user input!**
- Client-side validation: JavaScript and built-in form validation (done by the browser)
  - Define required fields and input formats.
- Required attribute
  - `<input type="text" required />`
- Pattern attribute: validate against a regular expression
  - `<input type="text" pattern="[A-Za-z]+" />`
- Min(length) & max(length) attributes
  - `<input type="number" min="5" max="25" />`
- Custom error messages have to be set up via JavaScript.
- Server-side validation (before data is processed)

Forms and UX

- Always add labels to input fields
  - Placeholders are not enough!
- Provide (do not remove!) focus states
- Care about the tab order for people using the keyboard only for navigation
- Provide helpful hints and error messages.
- Read more in: Forms on the Web That Make Me Cry and Why

Graphics

- <canvas>: Drawing area (drawing via JS possible)
  - https://www.w3schools.com/html/html5_canvas.asp
- <svg>: Container for SVG graphics (creation in standard SVG syntax possible)
  - https://www.w3schools.com/html/html5_svg.asp

Validators

- Check a document against a standard
- Quality assurance
  - ensures functionality/display in the most common browsers (see https://validator.w3.org/docs/why.html)
- Recommended validator
  - W3C: https://validator.w3.org/

Web Accessibility

- Web Accessibility Initiative (WAI)
- Web Content Accessibility Guidelines (WCAG)

"The power of the Web is in its universality. Access by everyone regardless of disability is an essential aspect."

Tim Berners Lee

Accessibility

- To make a product accessible means to build it in a way that people with disabilities can perceive, understand, navigate, and interact with it

- Accessible websites (and other software products) are not only optimized for blind, deaf or motor-impaired people, but for all of us!

- WHO: ~15% of all people have an impairment

What is WAI-ARIA? What is WCAG?

- WAI -> Web Accessibility Initiative
  - Develops standards and support materials to help you understand and implement accessibility
- ARIA -> Accessible Rich Internet Applications
  - A specification defining a set of additional HTML attributes that can be applied to elements to provide additional semantics and improve accessibility wherever it is lacking. (Source)
- WCAG is short for Web Content Accessibility Guidelines
  - Developed with a goal of providing a single shared standard for web content accessibility that meets the needs of individuals, organizations, and governments internationally. (Source)

WAI-ARIA

- A set of additional HTML attributes that can be applied to elements to provide additional semantics and improve accessibility wherever it is lacking.
  - **Roles** to define what an element is or does: search, tablist, navigation, banner ,…
  - **Properties** to provide additional meaning/semantics: aria-labelledby, aria-required, aria-live ,…
  - **States**: aria-disabled, aria-checked ,…
- The first rule of ARIA: don’t use it if there are native HTML elements and attributes providing the desired behavior!
  - **No ARIA is better than bad ARIA**
- Some examples, dos & don’ts

4 WCAG Principles

- **Perceivable** - Information and user interface components must be presentable to users in ways they can perceive.
- **Operable** - User interface components and navigation must be operable.
- **Understandable** - Information and the operation of the user interface must be understandable.
- **Robust** - Content must be robust enough that it can be interpreted by a wide variety of user agents, including assistive technologies.
- **3 Levels**: A, AA, AAA

Accessibility – Quick Rules

Accessibility – Quick Rules

- **Structure/content**: Use headings, titles, lists etc. as such (e.g. heading, not just large font with appropriate formatting)
- **Alt texts**: Alternative text for images (mandatory for HTML5 anyway)
- **Color**: sufficient contrast, not the sole information carrier
- **Navigation** support/accessibility via keyboard: accesskeys, focus & tabindex
- **Labels** for form elements

Use alternative text for icon buttons

- Alternative text for images, videos, and especially for buttons (or links) using icons only
  - Screenreader user is able to perceive what a button is for
  - When a button only contains an icon (or for example an “x” character to indicate it’s a button that closes a modal or similar), a screenreader would only read “button”, but not tell the user what the button does.
  - alt=“” -> tells screenreader to ignore image
- Use the aria-label attribute:

```
<button aria-label="Beitrag erstellen Dialog schließen">
  <svg ... />
</button>
```



- Difference between aria-label, aria-labelledby and aria-describedby

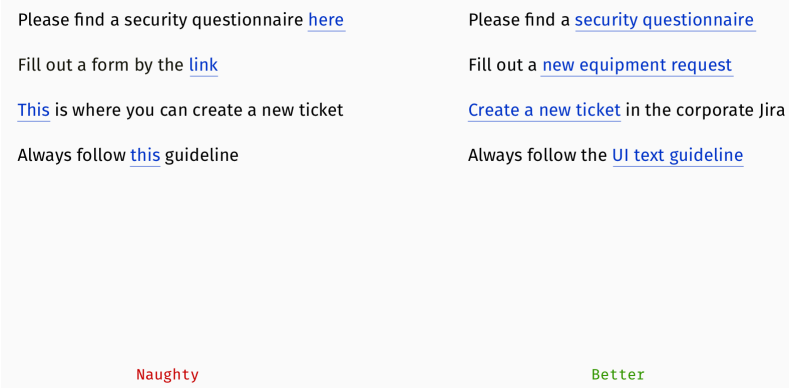
Use meaningful link text

- Instead of using code like this

```
<p>To read more about penguins <a
href="…">click here</a>.</p>
```

provide meaningful link texts to indicate what the user can expect when clicking the link:

```
<p>To learn more read this <a
href="…">article about penguins</a>.</p>
```



From this article: Designing Better Links For The Web on Smashing Magazine

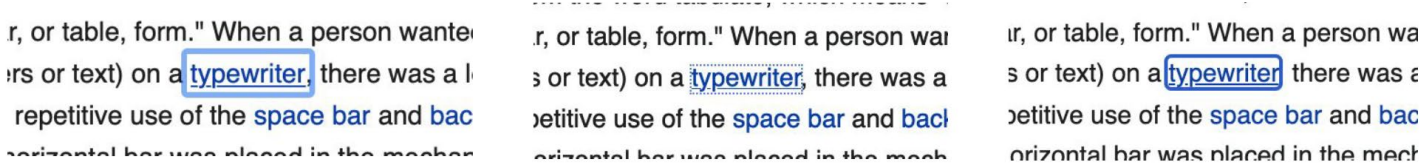
Use buttons and links instead of divs and spans

- Even worse than having a button (or link) without a label is using no <button> or <a> element at all!
  - Often or elements with a JavaScript click handler are used instead
- **Potential Problems:**
  - User can't interact with the button/link using the keyboard:
    - Element can't be focused.
    - The element is not part of the tab-order.
  - When the JS code does not work, the action when clicking the button is broken as well.

  - Find more problems and solutions in these two HTMHell articles:
    - https://www.htmhell.dev/22-the-good-ol-div-link/
    - https://www.htmhell.dev/20-close-buttons/

Never remove focus states

- When elements on a web page are focused they usually get an outline
- Sometimes devs/designers do not like this outline and add the following CSS code:
  - `*:focus { outline: 0; }`
  - This is very bad when no other way of highlighting focused elements is added instead
  - Users navigating by keyboard will be completely lost on the page because visual feedback is missing

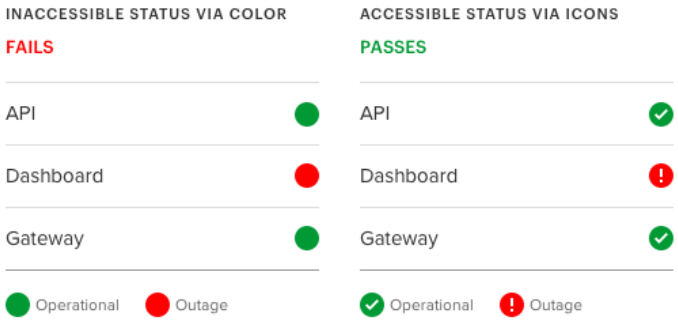


Tab order and visual order should be the same

- Some users prefer navigating through a web page using their keyboard
  - Use “tab” key (↹) to jump from one interactive element to the next
  - Only works when interactive elements (a, button, input, select,…) are provided by the developer.
- Be careful when the visual order of elements is not the same as the order in the DOM (which is the default tab order)!
  - Examples: using the order property in flexbox or grid, using flex-direction: row-reverse, or similar…
- The default tab order can be changed by using the tabindex attribute.

Do not rely on color alone

- It’s common to use green to indicate success and red to indicate errors
- Lot of people cannot distinguish these two colors!
- Make sure to not only rely on color alone when displaying errors or success messages





Images from https://uxmovement.com/buttons/the-myths-of-color-contrast-accessibility/

Let the browser help

- Many useful accessibility tools that can be found in Chrome's dev tools:
  - Inspect element tooltip
  - Contrast ratio checker
  - Lighthouse

